Authentication & Security — Token Issuance & The Verification Gap

InstantMudra Enterprises Admin Portal

Three token-issuing endpoints, one shared token format

/auth/login → /auth/verify-otp
OTP-verified, real password check
(BCrypt + plaintext fallback)
salt: server-random (SecureRandom)

/auth/verify-forgot-mpin-otp
Password-reset OTP flow
(same token shape as login)
salt: server-random (SecureRandom)

/api/aes/generate-token
OTP-less shortcut
(same password fallback)
⚠ salt: CLIENT-supplied in request body

fullToken = hierarchyId + "|" + salt + "|" + iv + "|" + AES-CBC(payload, key = PBKDF2(baseSecret, salt))

payload = generationTimestamp + "/" + expirationTimestamp + "/" + mobileNumber

AESFilter.doFilterInternal() — the real gatekeeper

(SecurityConfig permits anyRequest() regardless — see Module 1; only AESFilter's whitelist matters in practice)

path whitelisted? — matched by plain String.startsWith(), not exact path

split Authorization on "|" → [hierarchyCode, salt, iv, encryptedToken]

derive key = PBKDF2(baseSecret, salt-FROM-TOKEN, iterations, keyLength)

decrypt encryptedToken with derived key + iv-from-token → decrypted payload

split payload on "/" → check creds[0]/creds[1] (timestamps) only

⚠ hierarchyCode (the identity claim) is NEVER compared to the decrypted payload or anything else
⚠ creds[2] — the mobileNumber embedded in the payload — is decrypted but discarded, never checked

PASS — request authorized

⚠ CRITICAL — confirmed: identity spoofing via plaintext token prefix

Legitimate token, issued to a real Retailer:
`NFT-RT-045 | saltBase64 | ivBase64 | encryptedPayload`

Attacker edits only the plaintext label — salt / iv / ciphertext copied verbatim, unchanged:
`NFT-AD-001 | saltBase64 | ivBase64 | encryptedPayload`

→ AESFilter re-derives the SAME key (salt unchanged) → decrypts successfully (ciphertext untouched)
→ timestamps still valid → PASSES. TokenUtil.extractHierarchyId() now returns "NFT-AD-001".
Every downstream check that trusts this value (HierarchyAccessService's AD bypass, UserContext.isAdmin) now treats the caller as full Admin.

Any authenticated user, at any hierarchy level, can escalate to any other identity — including Admin — by editing plaintext in their own Authorization header. No key knowledge or server interaction required.

Other confirmed findings

Dead per-hierarchy salt infrastructure
AESKeyService.getSecretKeyForHierarchy() and a near-duplicate copy in AESController both maintain the same 8 hardcoded HIERARCHY_SALTS — neither is ever called. Every live path uses one shared baseSecret + an in-band, per-token salt instead.

Plaintext-equality password fallback
AuthController.handlePasswordValidation() (the /auth/login flow) and AESController.generateToken() both accept inputPassword.equals(storedPassword) as a pass condition alongside the real BCrypt check.

Unauthenticated encryption mode
AES/CBC/PKCS5Padding everywhere — no HMAC/GCM tag, so there is no integrity check over the ciphertext itself either, consistent with the spoofing gap above.

JwtUtil — fully dead code
A complete, correctly-implemented signed-JWT utility (jjwt, HS256) exists but has zero call sites anywhere in the codebase — confirmed by full-repo search.

Correction to Module 2
AESFilter matches its whitelist via plain String.startsWith(), not exact-path matching. "/api/hierarchies/tree" therefore DOES match "/api/hierarchies/tree-diagram" too — that JSON endpoint is also unauthenticated, not just /view and /data.

Standard step · Flagged risk · Critical — confirmed authentication bypass · Pass / allow outcome